

Algorithme k-NN

k-Nearest Neighbors

Application : Prédiction des exacerbations asthmatiques
(à partir du fichier TrainDataDC2.csv.gz)

Fatima Zahra SAAD
Nedah ASAS
Yolhadere DURIMEL
Mohammed MHAIMID

Plan de la présentation

01

Qu'est-ce que le kNN ?

L'idée intuitive derrière l'algorithme

02

Comment ça fonctionne ?

Étapes clés pas à pas

03

Notre contexte médical

Prédire les crises d'asthme

04

Démonstration du code

Pipeline

05

Limites

Pièges à éviter

01 · Qu'est-ce que le kNN ?

Le principe en une phrase :

"Pour classer un patient inconnu, je regarde quels patients connus lui ressemblent le plus, et je lui attribue la classe majoritaire parmi eux."

Intuition médicale :

Un médecin diagnostique un nouveau patient en le comparant à des cas similaires déjà connus.

kNN est un algorithme supervisé, non paramétrique, paresseux (lazy learner).

02 · Comment ça fonctionne ? — Les 4 étapes clés

Étape 1 · Représenter les données

Chaque patient devient un point dans un espace multidimensionnel (âge, adhérence, médicaments, comorbidités...)



Étape 2 · Calculer les distances

Pour chaque nouvel individu, on calcule sa distance (Euclidienne, Manhattan...)
à TOUS les individus déjà connus.



Étape 3 · Sélectionner les k voisins

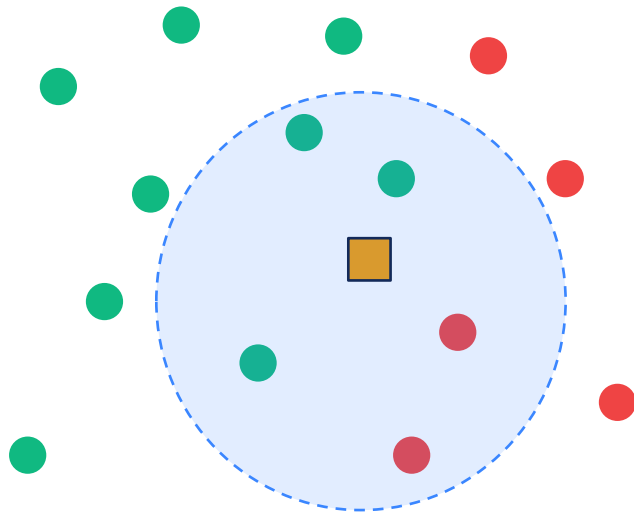
On retient les k points les plus proches.
k=1 → très sensible au bruit | k grand → frontière plus douce



Étape 4 · Vote majoritaire

Les k voisins votent pour leur classe.
La classe majoritaire est assignée au nouveau patient.

02 · Visualisation : Distance & Vote (k=5)



-  Patient sain (non-exacerbation)
-  Patient malade (exacerbation)
-  Nouveau patient à classer
-  Cercle de voisinage (k=5)

Vote des 5 voisins

3 × Sain
2 × Malade

→ Classé : **SAIN** (majorité)

03 · Notre contexte médical — Prédire les crises d'asthme

Objectif : prédire si un patient asthmatique fera une exacerbation dans l'année à venir
- Fichier trainDataDC2.csv.gz

Variable cible

0 → Aucune exacerbation
1 → Au moins 1 exacerbation

post_index_exacerbations365 > 0

Variables prédictives (features)

Numériques

Âge, adhérence au traitement,
nombre de canistres, coûts
soins

Catégorielles

Sexe, médicaments
précédents,
comorbidités (rhinite,
sinusite...)

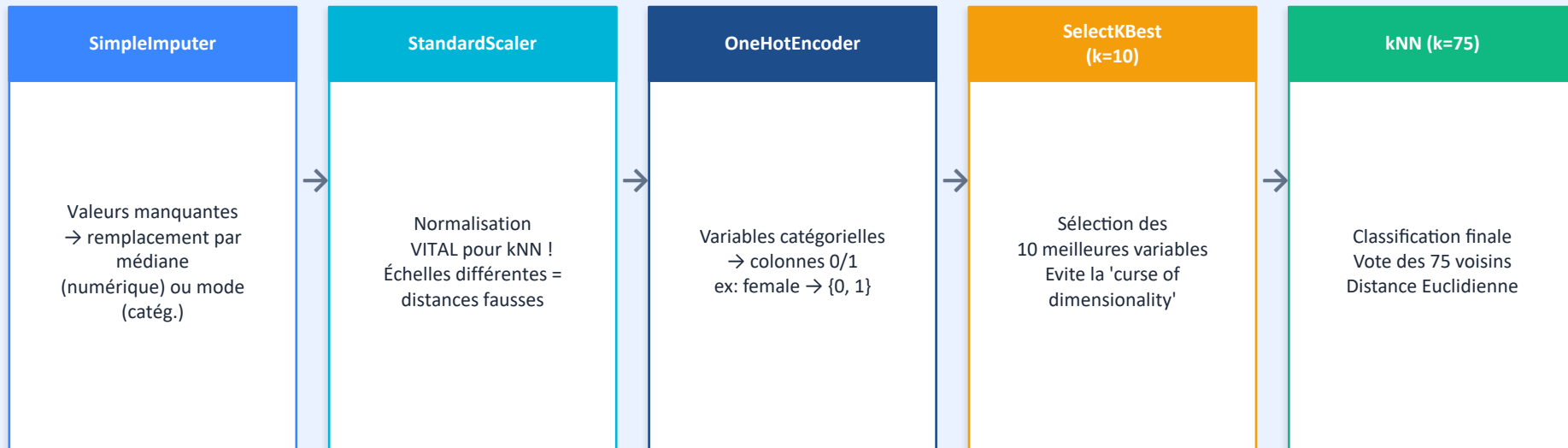
Supprimées

Identifiant patient, variables
'du futur' (fuite de données !)

Défi : déséquilibre des classes

La majorité des patients n'ont PAS d'exacerbation. Si on ne fait rien, le modèle prédit tout le temps "Sain" et obtient une bonne précision... mais rate tous les malades !

04 · Code — Pipeline de traitement des données



Extrait du code

```
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('feature_selection', SelectKBest(score_func=f_classif, k=10)),
    ('classifier', KNeighborsClassifier(n_neighbors=75, weights='uniform', p=2))
])
```

04 · Code — Découpage des données & rééquilibrage

Découpage stratifié des données



▲ Seuil optimal

▲ Évaluation finale

Rééquilibrage par oversampling (clonage des malades)

Problème : Si 80% des patients sont sains, kNN votera toujours 'Sain' → il rate les malades !

Solution : On clone aléatoirement les patients malades jusqu'à équilibrer les deux classes.

⚠ Uniquement sur le jeu Train, jamais sur Validation ni Test !

04 · Code — Seuil de décision & Résultats

Optimisation du seuil de décision

Par défaut, sklearn classe en '1' si > 50% des voisins votent malade.

Mais avec l'oversampling, le modèle tend à sur-prédire les malades.

→ On teste des seuils entre 0.50 et 0.72 sur le set de Validation

→ On garde le seuil qui maximise le F1-score.

```
for seuil in np.arange(0.50, 0.72, 0.02):  
    preds = (probas_val >= seuil).astype(int)  
    score = f1_score(y_val, preds)  
    if score > meilleur_f1:  
        meilleur_seuil = seuil
```

Résultats obtenus

F1-Score **~0.20**

Précision **~0.13**

Seuil optimal **0.52**

F1 = 0.20 en médecine

Ce score peut sembler faible, mais c'est réaliste : on ne peut pas prédire l'asthme à 100%. L'objectif est d'isoler une population à risque pour orienter les soins.

05 · Limites du kNN — Ce qu'il faut savoir

Lenteur à la prédiction

kNN ne crée pas de modèle appris : il compare le nouveau patient à TOUS les patients d'entraînement à chaque prédiction. Sur des millions de points → très lent.

Sensible à l'échelle

Si une variable est en milliers (€) et une autre en dizaines (âge), la première domine le calcul de distance. Le StandardScaler est OBLIGATOIRE.

Curse of Dimensionality

Avec trop de variables, toutes les distances deviennent similaires → le concept de 'voisin proche' perd son sens. D'où l'usage de SelectKBest.

Sensible au déséquilibre

kNN vote à la majorité. Si 90% des données sont 'Sain', il votera toujours Sain. Nécessite un rééquilibrage (oversampling).

Choix de k critique

k trop petit → sur-apprentissage (bruit). k trop grand → sous-apprentissage (frontières floues). Ici k=75 est un grand voisinage stabilisateur.

Consomme de la mémoire

Tout le dataset d'entraînement doit rester en mémoire (lazy learner). Incompatible avec des systèmes embarqués ou des bases très volumineuses.

Conclusion

L'algorithme

kNN classe par similarité : les k voisins les plus proches votent. Simple, intuitif, mais lazy (pas de modèle appris).

Notre pipeline

Imputation → Normalisation → Encodage
→ Sélection de features (k=10) → kNN
(k=75) + oversampling + seuil optimisé.

Le résultat

F1 $\approx$ 0.20 sur patients asthmatiques.
Résultat réaliste en médecine : on détecte
une population à risque, pas une certitude.

*En santé publique, un modèle imparfait qui identifie 20% de vrais malades supplémentaires
peut avoir un impact clinique réel sur des millions de patients.*